CHƯƠNG 5

# Danh sách liên kết, Ngăn xếp, Hàng đợi

Bốn chương đầu tập trung vào thuật toán, và cấu trúc dữ liệu duy nhất được dùng là mảng. Từ chương này trở đi trọng tâm chuyển sang cấu trúc dữ liệu: cách tổ chức dữ liệu trong bộ nhớ để các thao tác cần thiết chạy nhanh.

Xuất phát điểm là một nhận xét về mảng. Mảng cho phép truy cập phần tử thứ $i$ trong $O(1)$, vì địa chỉ của nó tính được trực tiếp từ địa chỉ đầu mảng. Đổi lại, việc chèn hoặc xoá một phần tử ở giữa buộc phải dịch chuyển toàn bộ phần đuôi, tốn $O(n)$. Ba cấu trúc trong chương này giải quyết đúng nhược điểm đó.

## 5.1 Kiểu dữ liệu trừu tượng

Trước khi bàn từng cấu trúc cụ thể, cần một khái niệm giúp tách bạch hai câu hỏi vẫn hay bị trộn lẫn: *cấu trúc này làm được những gì* và *nó được xây bằng cái gì*.

**Định nghĩa 5.1.** (*Kiểu dữ liệu trừu tượng*) **Kiểu dữ liệu trừu tượng** (*abstract data type*, viết tắt ADT) là một mô hình toán học gồm hai phần: tập giá trị mà kiểu đó có thể nhận, và tập các *thao tác* thực hiện được trên nó cùng đặc tả của từng thao tác. Định nghĩa này *không* nói cấu trúc được cài đặt thế nào trong bộ nhớ.

Ngăn xếp là một ví dụ. Đặc tả của nó gồm bốn thao tác — đưa vào, lấy ra, xem đỉnh, kiểm tra rỗng — cùng một luật: phần tử lấy ra luôn là phần tử vào sau cùng. Đặc tả đó không hề nhắc tới mảng hay con trỏ. Mục 5.4 sẽ cho thấy cùng một đặc tả ấy cài đặt được bằng mảng và cũng cài đặt được bằng danh sách liên kết, với cùng độ phức tạp.

Việc phân biệt hai tầng có ba lợi ích thực tế.

1. **Thay cài đặt mà không sửa chương trình dùng nó.** Nếu chương trình chỉ gọi bốn thao tác trong đặc tả, ta đổi cách cài đặt bên trong mà phần còn lại không cần biết.
2. **Lập luận về tính đúng đắn ở tầng đặc tả.** Chứng minh "thuật toán kiểm tra ngoặc là đúng" chỉ cần luật vào sau ra trước, không cần biết ngăn xếp được cài bằng gì.
3. **Chọn cài đặt theo yêu cầu về hiệu năng.** Cùng một ADT danh sách, bản cài bằng mảng và bản cài bằng liên kết có bảng độ phức tạp trái ngược nhau; việc chọn bản nào là quyết định kỹ thuật, không phải quyết định về mặt logic.

**Nhận xét 5.1.** Trong thư viện chuẩn của C++, sự phân tầng này được thể hiện tường minh: hai lớp `std::stack` và `std::queue` không phải cấu trúc dữ liệu mà là *container adaptor*: chúng chỉ định nghĩa tập thao tác và uỷ thác việc lưu trữ cho một cấu trúc bên dưới, mặc định là `std::deque`. Người dùng có thể yêu cầu dùng cấu trúc khác, chẳng hạn `stack<int, vector<int>>`, mà không phải sửa dòng nào khác.

**Bảng 5.1.** Các kiểu dữ liệu trừu tượng trong chương này và các cài đặt của chúng.

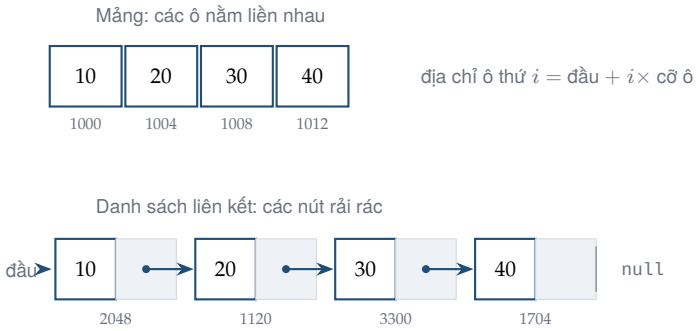
| ADT | Thao tác trong đặc tả | Các cài đặt trình bày trong chương |
|---|---|---|
| Danh sách | truy cập vị trí $i$, tìm, chèn, xoá | mảng; danh sách liên kết đơn; danh sách liên kết kép |
| Ngăn xếp | đưa vào, lấy ra, xem đỉnh, kiểm tra rỗng | mảng động; danh sách liên kết đơn |
| Hàng đợi | thêm cuối, lấy đầu, kiểm tra rỗng | mảng vòng; danh sách liên kết đơn có con trỏ tới nút cuối |

## 5.2 Danh sách và hai cách cài đặt

**Định nghĩa 5.2.** (*Danh sách*) **Danh sách** là một dãy hữu hạn các phần tử cùng kiểu, có thứ tự. Các thao tác cơ bản gồm: truy cập phần tử thứ $i$, tìm kiếm theo giá trị, chèn một phần tử vào vị trí bất kỳ, xoá một phần tử.

Có hai cách cài đặt danh sách.

- **Cài đặt bằng mảng**, còn gọi là danh sách kề: các phần tử nằm liên tiếp trong bộ nhớ.
- **Cài đặt bằng liên kết**: mỗi phần tử nằm ở một vùng nhớ riêng và lưu thêm địa chỉ của phần tử kế tiếp.




**Hình 5.1.** Hai cách cài đặt danh sách. Số nhỏ màu xám là địa chỉ ô nhớ.

### 5.2.1 So sánh hai cách

**Bảng 5.2.** Độ phức tạp các thao tác trên hai cách cài đặt danh sách.

| Thao tác | Mảng | Danh sách liên kết |
|---|---|---|
| Truy cập phần tử thứ $i$ | $O(1)$ | $O(n)$ |
| Tìm theo giá trị | $O(n)$ | $O(n)$ |
| Chèn vào đầu | $O(n)$ | $O(1)$ |
| Chèn sau một nút đã biết địa chỉ | $O(n)$ | $O(1)$ |
| Xoá một nút đã biết địa chỉ | $O(n)$ | $O(1)$ |
| Bộ nhớ cho mỗi phần tử | chỉ dữ liệu | dữ liệu + con trỏ |
| Cần biết trước số phần tử | có | không |

Bảng trên là lý thuyết. Số đo dưới đây cho thấy khác biệt trong thực tế, đo bằng C++ biên dịch với `-O2`.

**Bảng 5.3.** Thời gian chèn $n$ phần tử vào *đầu* danh sách, đơn vị mi-li-giây.

| $n$ | `vector` | Danh sách liên kết | Tỉ lệ |
|---|---|---|---|
| 10 000 | 2,1 | 0,3 | 7 lần |
| 50 000 | 80,2 | 1,2 | 67 lần |
| 200 000 | 1 328,8 | 5,2 | 255 lần |

Ở chiều ngược lại, với $n = 200\,000$ phần tử và 1000 lần truy cập ngẫu nhiên theo chỉ số, mảng mất 0,009 mi-li-giây còn danh sách liên kết mất 269,8 mi-li-giây, chậm hơn khoảng ba mươi nghìn lần. Hai bảng số liệu này là lý do không có cấu trúc nào "tốt hơn" cấu trúc nào: lựa chọn phụ thuộc vào thao tác nào được thực hiện nhiều nhất.

**Một yếu tố không xuất hiện trong ký hiệu $O$.** Chênh lệch 30 000 lần ở phép truy cập ngẫu nhiên lớn hơn nhiều so với dự đoán từ $O(1)$ và $O(n)$. Nguyên nhân là bộ nhớ đệm của bộ xử lý: các ô của mảng nằm liền nhau nên một lần đọc bộ nhớ nạp luôn nhiều ô kế tiếp, còn các nút của danh sách liên kết rải rác nên mỗi bước nhảy là một lần truy cập bộ nhớ chính. Ký hiệu $O$ bỏ qua yếu tố này, nhưng trong chương trình thực tế nó thường quyết định.

### 5.2.2 Mảng động và phân tích khấu hao

Mảng thường có kích thước cố định từ lúc khai báo. Các kiểu `vector` của C++, `list` của Python và `Vector` của Julia là **mảng động**: chúng cho phép thêm phần tử vào cuối mà không cần biết trước số phần tử. Câu hỏi là thao tác thêm đó tốn bao nhiêu.

**Cơ chế.** Mảng động giữ hai đại lượng: số phần tử đang có $n$ và **sức chứa** $c \geq n$, tức số ô đã cấp phát. Khi thêm mà $n < c$, chỉ việc ghi vào ô trống kế tiếp, tốn $O(1)$. Khi $n = c$, phải cấp phát một vùng nhớ mới, chép toàn bộ $n$ phần tử sang, rồi giải phóng vùng cũ; lần thêm đó tốn $O(n)$.

Nhìn riêng lần thêm gây cấp phát lại thì chi phí là $O(n)$, nên nếu đánh giá theo trường hợp xấu nhất của *một* thao tác, ta phải kết luận thêm phần tử tốn $O(n)$. Kết luận đó đúng nhưng bi quan quá mức, vì trường hợp xấu nhất không thể xảy ra liên tiếp. Công cụ đánh giá đúng cho tình huống này là phân tích khấu hao.

**Định nghĩa 5.3.** (*Chi phí khấu hao*) Xét một dãy $m$ thao tác liên tiếp trên một cấu trúc dữ liệu. **Chi phí khấu hao** của một thao tác là tổng chi phí của cả dãy chia cho $m$. Nói cách khác, đó là chi phí trung bình trên mỗi thao tác trong *trường hợp xấu nhất của cả dãy*, chứ không phải trung bình theo một phân bố xác suất nào.

**Khấu hao khác trường hợp trung bình.** Hai khái niệm này hay bị lẫn. *Trường hợp trung bình* lấy kỳ vọng theo một phân bố xác suất giả định trên dữ liệu vào, nên kết luận chỉ đúng nếu phân bố giả định đúng. *Chi phí khấu hao* là một bảo đảm chắc chắn về tổng chi phí của cả dãy thao tác, không giả định gì về dữ liệu và không có yếu tố ngẫu nhiên.

**Định lý 5.1.** (*Chi phí khấu hao của thao tác thêm vào cuối*) *Nếu mỗi lần đầy thì sức chứa được nhân đôi, thì $n$ lần thêm liên tiếp vào một mảng động rỗng tốn tổng cộng $O(n)$ thời gian. Chi phí khấu hao của mỗi lần thêm là $O(1)$.*

*Chứng minh.* Chi phí gồm hai phần: chi phí ghi giá trị, tổng cộng đúng $n$; và chi phí chép dữ liệu khi cấp phát lại.

Việc cấp phát lại xảy ra khi số phần tử đạt $1, 2, 4, 8, \ldots$, tức tại các luỹ thừa của 2 không vượt quá $n$. Lần cấp phát lại tại mốc $2^k$ phải chép $2^k$ phần tử. Vậy tổng chi phí chép là

$$\sum_{k=0}^{\lfloor \log_2 n \rfloor} 2^k = 2^{\lfloor \log_2 n \rfloor + 1} - 1 < 2n.$$

Ở đây ta dùng công thức tổng cấp số nhân $1 + 2 + 4 + \cdots + 2^K = 2^{K+1} - 1$.

Tổng chi phí của cả dãy nhỏ hơn $n + 2n = 3n$, tức $O(n)$; chia cho $n$ thao tác được $O(1)$ cho mỗi thao tác. ∎

**Nhận xét 5.2.** [Vì sao phải nhân đôi chứ không phải cộng thêm một hằng số] Giả sử thay vì nhân đôi, mỗi lần đầy ta chỉ tăng sức chứa thêm một hằng số $b$. Khi đó việc cấp phát lại xảy ra tại các mốc $b, 2b, 3b, \ldots, n$, và tổng chi phí chép là

$$b + 2b + \cdots + n = \frac{n}{b} \cdot \frac{b + n}{2} = \Theta(n^2).$$

Chi phí khấu hao trở thành $\Theta(n)$ chứ không còn là $O(1)$. Việc nhân sức chứa với một hệ số lớn hơn 1 — các thư viện thực tế dùng hệ số 2 hoặc 1,5 — là điều kiện để tổng chi phí chép là cấp số nhân và do đó bị chặn bởi một bội của $n$.

> **Hệ quả thực tế.** Nếu biết trước số phần tử, nên báo cho mảng động biết để nó cấp phát một lần duy nhất: `a.reserve(n)` trong C++, `sizehint!(a, n)` trong Julia. Việc này không đổi độ phức tạp nhưng loại bỏ hoàn toàn các lần chép lại, và trên dữ liệu lớn thường rút ngắn thời gian chạy vài chục phần trăm.

## 5.3 Danh sách liên kết đơn

### 5.3.1 Cấu trúc nút

Mỗi nút gồm hai phần: phần dữ liệu và phần liên kết trỏ tới nút kế tiếp. Nút cuối cùng có liên kết rỗng.

JULIA

```julia
mutable struct Nut
    gt::Int
    ke::Union{Nut, Nothing}      # nút kế tiếp, hoặc nothing nếu là nút cuối
end
Nut(gt) = Nut(gt, nothing)

mutable struct DSLK
    dau::Union{Nut, Nothing}
    n::Int
end
DSLK() = DSLK(nothing, 0)
```

PYTHON

```python
class Nut:
    __slots__ = ("gt", "ke")            # tiết kiệm bộ nhớ

    def __init__(self, gt, ke=None):
        self.gt = gt
        self.ke = ke                    # None nếu là nút cuối

class DSLK:
    def __init__(self):
        self.dau = None
        self.n = 0
```

C++

```cpp
struct Nut {
    int gt;
    Nut *ke;
    Nut(int g, Nut *k = nullptr) : gt(g), ke(k) {}
};

struct DSLK {
    Nut *dau = nullptr;
    int n = 0;
};
```

**Ba cách diễn đạt cùng một ý.** C++ dùng *con trỏ* `Nut *`, tức biến chứa địa chỉ ô nhớ, và giá trị rỗng là `nullptr`. Python và Julia không có con trỏ tường minh: biến `ke` tham chiếu trực tiếp tới đối tượng nút, và giá trị rỗng là `None` hoặc `nothing`. Về mặt logic ba cách là một. Khác biệt thực tế là C++ yêu cầu người viết tự giải phóng bộ nhớ bằng `delete`, còn hai ngôn ngữ kia có bộ thu gom rác làm việc đó.
Trong Julia, phải so sánh bằng `===` và `!==` khi kiểm tra `nothing`, không dùng `==`.

### 5.3.2 Duyệt danh sách

Mọi thao tác đều bắt đầu bằng việc đi từ nút đầu theo các liên kết.

GIẢ MÃ

```
DUYET(dau)
1  p ← dau
2  while p ≠ null
3      xử lý p.gt
4      p ← p.ke
```

JULIA

```julia
function in_danh_sach(L::DSLK)
    p = L.dau
    while p !== nothing
        print(p.gt, " ")
        p = p.ke
    end
    println()
end
```

PYTHON

```python
def in_danh_sach(L):
    p = L.dau
    while p is not None:
        print(p.gt, end=" ")
        p = p.ke
    print()
```

C++

```cpp
void in_danh_sach(const DSLK &L) {
    for (Nut *p = L.dau; p != nullptr; p = p->ke)
        cout << p->gt << " ";
    cout << "\n";
}
```

### 5.3.3 Thêm phần tử

#### 5.3.3.1 Thêm vào đầu

Đây là thao tác rẻ nhất: tạo nút mới, cho nó trỏ vào nút đầu cũ, rồi cập nhật con trỏ đầu.

JULIA

```julia
function them_dau!(L::DSLK, gt)
    L.dau = Nut(gt, L.dau)
    L.n += 1
    return L
end
```

PYTHON

```python
def them_dau(L, gt):
    L.dau = Nut(gt, L.dau)
    L.n += 1
```

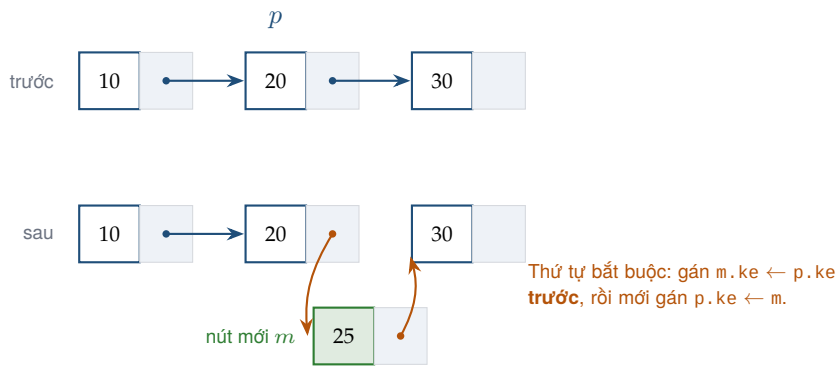
C++

```cpp
void them_dau(DSLK &L, int gt) {
    L.dau = new Nut(gt, L.dau);
    L.n++;
}
```

#### 5.3.3.2 Chèn sau một nút

> **Bài toán.** Cho danh sách liên kết đơn và một nút $p$ đã biết địa chỉ. Chèn một nút mới mang giá trị `gt` vào ngay sau $p$, trong thời gian $O(1)$.

**Ý tưởng.** Toàn bộ danh sách được giữ bởi các liên kết, nên chèn một nút nghĩa là sửa lại đúng hai liên kết: liên kết của nút mới phải trỏ vào phần đuôi cũ, và liên kết của $p$ phải trỏ vào nút mới. Không phần tử nào phải dịch chuyển, khác hẳn với mảng.




**Hình 5.2.** Chèn nút mới sau nút $p$. Đổi thứ tự hai phép gán sẽ làm mất phần đuôi của danh sách.

**Thuật toán 5.1.** *Chèn một nút vào sau nút đã biết*

1. Tạo nút mới $m$ mang giá trị `gt`.
2. Cho $m$ trỏ vào nút mà $p$ đang trỏ tới, tức phần đuôi hiện tại.
3. Cho $p$ trỏ vào $m$.
4. Tăng biến đếm số phần tử lên 1.

Hai bước 2 và 3 *không được đổi thứ tự*.

**GIẢ MÃ**

```
1  CHEN-SAU(p, gt)
2  1  m ← nút mới với giá trị gt
3  2  m.ke ← p.ke          // phải làm bước này TRƯỚC
4  3  p.ke ← m
```

### Giải thích từng bước.

| Dòng | Ý nghĩa và lý do có dòng này |
|---|---|
| 1 | Cấp phát một nút mới và ghi giá trị vào phần dữ liệu. Ở bước này phần liên kết của $m$ chưa có nghĩa. |
| 2 | Nối $m$ vào phần đuôi. Sau dòng này, phần đuôi cũ được hai nút cùng trỏ tới: $p$ và $m$. Danh sách tạm thời chưa hợp lệ nhưng chưa mất gì. |
| 3 | Cắt liên kết cũ của $p$ và trỏ nó sang $m$. Vì phần đuôi đã được $m$ giữ ở dòng 2 nên việc cắt này an toàn. |
| 2–3 | *Nếu đảo thứ tự hai dòng*: sau dòng 3 chạy trước, `p.ke` đã bằng $m$; khi đó dòng 2 gán `m.ke` ← `p.ke` $= m$, tức nút mới trỏ vào chính nó. Toàn bộ phần đuôi không còn ai trỏ tới và bị mất, và mọi vòng lặp duyệt về sau chạy vô hạn. |

**JULIA**

```julia
function chen_sau!(L::DSLK, p::Nut, gt)
    m = Nut(gt, p.ke)       # m.ke = p.ke ngay khi tạo
    p.ke = m
    L.n += 1
```

```
    return m
end
```

PYTHON

```python
def chen_sau(L, p, gt):
    m = Nut(gt, p.ke)       # m.ke = p.ke ngay khi tạo
    p.ke = m
    L.n += 1
    return m
```

C++

```cpp
Nut *chen_sau(DSLK &L, Nut *p, int gt) {
    Nut *m = new Nut(gt, p->ke);    // m->ke = p->ke ngay khi tạo
    p->ke = m;
    L.n++;
    return m;
}
```

**Lỗi kinh điển.** Nếu viết `p.ke = m` trước rồi mới `m.ke = p.ke`, thì ở dòng thứ hai `p.ke` đã bằng `m`, nên `m.ke = m`: nút mới trỏ vào chính nó. Toàn bộ phần đuôi của danh sách bị mất, và mọi vòng lặp duyệt sau đó sẽ chạy vô hạn. Chương trình không báo lỗi, nó chỉ treo.

#### 5.3.3.3 Thêm vào cuối

Không có con trỏ tới nút cuối thì phải duyệt cả danh sách, tốn $O(n)$.

PYTHON

```python
def them_cuoi(L, gt):
    m = Nut(gt)
    if L.dau is None:
        L.dau = m
    else:
        p = L.dau
        while p.ke is not None:     # đi tới nút cuối
            p = p.ke
        p.ke = m
    L.n += 1
```

C++

```cpp
void them_cuoi(DSLK &L, int gt) {
    Nut *m = new Nut(gt);
    if (L.dau == nullptr) L.dau = m;
    else {
        Nut *p = L.dau;
        while (p->ke != nullptr) p = p->ke;      // đi tới nút cuối
        p->ke = m;
    }
    L.n++;
}
```

JULIA

```julia
function them_cuoi!(L::DSLK, gt)
    m = Nut(gt)
    if L.dau === nothing
        L.dau = m
    else
        p = L.dau
        while p.ke !== nothing
            p = p.ke
        end
        p.ke = m
    end
    L.n += 1
    return L
end
```

**Nhận xét 5.3.** Nếu chương trình thêm vào cuối thường xuyên, nên giữ thêm một con trỏ `cuoi` trỏ vào nút cuối cùng. Khi đó thao tác thêm vào cuối chỉ tốn $O(1)$, đổi lại mọi thao tác xoá phải nhớ cập nhật con trỏ đó.

### 5.3.4 Xoá phần tử

Muốn xoá một nút, phải biết nút *đứng trước* nó, vì liên kết chỉ đi một chiều. Vì vậy vòng lặp tìm kiếm phải giữ hai con trỏ.




**Hình 5.3.** Xoá nút $p$ khỏi danh sách liên kết đơn: nối nút đứng trước thẳng vào nút đứng sau.

> **Bài toán.** Cho danh sách liên kết đơn $L$ và một giá trị `gt`. Xoá nút đầu tiên mang giá trị đó, trả về *đúng* nếu tìm thấy và *sai* nếu không.

**Ý tưởng.** Muốn tách một nút ra khỏi danh sách, phải cho nút đứng trước nó trỏ thẳng sang nút đứng sau nó. Vì liên kết chỉ đi một chiều, từ một nút không có cách nào quay lại nút đứng trước; do đó vòng lặp tìm kiếm phải mang theo hai con trỏ, một trỏ vào nút đang xét và một trỏ vào nút liền trước. Trường hợp xoá nút đầu phải xử lý riêng, vì nút đầu không có nút đứng trước.

**Thuật toán 5.2.** *Xoá nút theo giá trị khỏi danh sách liên kết đơn*

1. Đặt `truoc` $\leftarrow$ rỗng và $p \leftarrow$ nút đầu.
2. Trong khi $p$ khác rỗng và giá trị của $p$ khác `gt`: dời cả hai con trỏ sang phải một bước.
3. Nếu $p$ rỗng thì không có giá trị cần xoá; trả về *sai*.
4. Nếu `truoc` rỗng thì $p$ chính là nút đầu; cho con trỏ đầu của danh sách trỏ sang nút kế tiếp của $p$.
5. Ngược lại, cho `truoc` trỏ sang nút kế tiếp của $p$.
6. Giải phóng $p$, giảm biến đếm, trả về *đúng*.

GIẢ MÃ

```
XOA(L, gt)
1  truoc ← null;  p ← L.dau
2  while p ≠ null and p.gt ≠ gt
3      truoc ← p;  p ← p.ke
4  if p = null
5      return SAI                    // không có giá trị này
6  if truoc = null
7      L.dau ← p.ke                  // xoá nút đầu
8  else
9      truoc.ke ← p.ke
10 giải phóng p
11 return ĐÚNG
```

### Giải thích từng bước.

| Dòng | Ý nghĩa và lý do có dòng này |
|---|---|
| 1 | Khởi tạo hai con trỏ. Giá trị rỗng của `truoc` mang một ý nghĩa riêng: "chưa đi qua nút nào", và về sau được dùng làm dấu hiệu nhận biết trường hợp xoá nút đầu. |
| 2 | Điều kiện lặp phải kiểm tra $p \neq$ `null` *trước* rồi mới đọc `p.gt`. Đảo thứ tự hai vế sẽ đọc trường dữ liệu của một con trỏ rỗng và chương trình dừng đột ngột. Tính chất short-circuit ở mục 1.1.9 là thứ bảo đảm điều này. |
| 3 | Dời cả hai con trỏ. Phải gán `truoc` $\leftarrow p$ *trước* khi dời $p$; đảo lại thì `truoc` và $p$ luôn trỏ vào cùng một nút. |
| 4–5 | Không tìm thấy. Trả về sớm để phần dưới không phải xét trường hợp $p$ rỗng nữa. |
| 6–7 | Trường hợp xoá nút đầu. Đây là trường hợp duy nhất phải sửa chính con trỏ đầu của danh sách chứ không sửa liên kết của một nút nào. Quên nhánh này là lỗi phổ biến nhất khi cài đặt thao tác xoá. |
| 8–9 | Trường hợp chung: nối nút trước thẳng vào nút sau, nhờ đó $p$ bị tách khỏi danh sách. |
| 10 | Chỉ có ý nghĩa trong C++; hai ngôn ngữ kia có bộ thu gom rác. Phải giải phóng *sau* khi đã đọc xong `p.ke` ở dòng 7 hoặc 9. |

PYTHON

```python
def xoa(L, gt):
    truoc, p = None, L.dau
    while p is not None and p.gt != gt:
        truoc, p = p, p.ke
    if p is None:
        return False                # không tìm thấy
    if truoc is None:
        L.dau = p.ke                # xoá nút đầu
    else:
        truoc.ke = p.ke
    L.n -= 1
    return True
```

C++

```cpp
bool xoa(DSLK &L, int gt) {
    Nut *truoc = nullptr, *p = L.dau;
    while (p != nullptr && p->gt != gt) { truoc = p; p = p->ke; }
    if (p == nullptr) return false;          // không tìm thấy
    if (truoc == nullptr) L.dau = p->ke;     // xoá nút đầu
    else                  truoc->ke = p->ke;
    delete p;                                // bắt buộc, tránh rò bộ nhớ
    L.n--;
    return true;
}
```

JULIA

```julia
function xoa!(L::DSLK, gt)
    truoc = nothing
    p = L.dau
    while p !== nothing && p.gt != gt
        truoc = p
        p = p.ke
    end
    p === nothing && return false
    if truoc === nothing
        L.dau = p.ke
    else
        truoc.ke = p.ke
    end
    L.n -= 1
    return true
end
```

**Hai chi tiết bắt buộc trong C++.** Thứ nhất, phải gọi `delete p` sau khi tách nút ra khỏi danh sách; quên là rò bộ nhớ. Thứ hai, phải gọi *sau* khi đã dùng xong `p->ke`; nếu viết `delete p` trước rồi mới đọc `p->ke`, chương trình đọc vùng nhớ đã giải phóng và cho kết quả không xác định.

### 5.3.5 Đảo ngược danh sách

Đây là bài tập kinh điển và cũng là câu hỏi phỏng vấn phổ biến. Thuật toán dùng ba con trỏ và đi đúng một lượt.

**Bài toán.** Cho danh sách liên kết đơn. Đảo ngược thứ tự các phần tử, dùng $O(1)$ bộ nhớ phụ và không cấp phát nút mới.

**Ý tưởng.** Đảo danh sách nghĩa là đảo chiều mọi liên kết: nút đang trỏ sang phải phải quay sang trỏ về trái. Đi một lượt từ đầu tới cuối, tại mỗi nút ta đảo chiều liên kết của nó. Khó khăn duy nhất là: ngay khi đảo chiều liên kết của một nút, ta mất đường đi tới phần còn lại của danh sách. Vì vậy phải lưu trước địa chỉ nút kế tiếp vào một biến thứ ba.

**Thuật toán 5.3.** *Đảo ngược danh sách liên kết đơn*

1. Đặt `truoc` ← rỗng và $p$ ← nút đầu.
2. Trong khi $p$ khác rỗng:
   (a) Lưu `sau` ← nút kế tiếp của $p$.
   (b) Cho $p$ trỏ ngược về `truoc`.
   (c) Dời `truoc` ← $p$ và $p$ ← `sau`.
3. Cho con trỏ đầu của danh sách trỏ vào `truoc`.

GIẢ MÃ

```
DAO(L)
1  truoc ← null;  p ← L.dau
2  while p ≠ null
3      sau ← p.ke        // giữ lại phần đuôi trước khi cắt
4      p.ke ← truoc      // đảo chiều liên kết
5      truoc ← p
6      p ← sau
7  L.dau ← truoc
```

**Giải thích từng bước.**

| Dòng | Ý nghĩa và lý do có dòng này |
|---|---|
| 1 | `truoc` bắt đầu bằng rỗng, vì nút đầu của danh sách cũ sẽ trở thành nút *cuối* của danh sách mới, và nút cuối phải có liên kết rỗng. |
| 3 | Biến `sau` là bắt buộc. Dòng 4 ngay sau đó ghi đè `p.ke`; không lưu lại trước thì không còn cách nào đi tiếp và thuật toán dừng ngay ở nút thứ hai. |
| 4 | Dòng thực sự làm việc: đảo chiều đúng một liên kết. |
| 5–6 | Dịch cả hai con trỏ sang phải một bước để lặp lại với nút kế tiếp. |
| 7 | Khi thoát vòng lặp thì $p$ rỗng, còn `truoc` đang trỏ vào nút cuối cùng đã xử lý, tức nút cuối của danh sách cũ. Đó chính là nút đầu của danh sách mới. |

**Định lý 5.2.** (*Tính đúng đắn của thuật toán đảo danh sách*) *Sau khi thuật toán kết thúc, danh sách chứa đúng các phần tử cũ theo thứ tự ngược lại, và không nút nào bị mất hay bị nhân đôi.*

*Chứng minh.* **Bất biến.** Trước mỗi lần lặp, danh sách ban đầu $x_1, x_2, \dots, x_n$ được chia làm hai phần rời nhau: `truoc` là đầu của danh sách $x_k, x_{k-1}, \dots, x_1$ đã đảo xong, còn $p$ là đầu của danh sách $x_{k+1}, \dots, x_n$ còn nguyên chiều cũ.

*Khởi tạo.* Với $k = 0$, phần đã đảo là danh sách rỗng, đúng bằng giá trị của `truoc`; phần còn lại là toàn bộ danh sách, đúng bằng giá trị của $p$.

*Duy trì.* Giả sử bất biến đúng với $k$. Dòng 3 giữ lại $x_{k+2}$. Dòng 4 cho $x_{k+1}$ trỏ về $x_k$, nên $x_{k+1}$ trở thành đầu của danh sách $x_{k+1}, x_k, \dots, x_1$. Hai dòng 5–6 đặt `truoc` $= x_{k+1}$ và $p = x_{k+2}$, tức bất biến đúng với $k + 1$.

*Kết thúc.* Vòng lặp dừng khi $p$ rỗng, tức $k = n$. Khi đó `truoc` là đầu của danh sách $x_n, \dots, x_1$, và

dòng 7 gán nó làm nút đầu. Trong suốt quá trình, mỗi nút được thăm đúng một lần và không có lệnh cấp phát hay giải phóng nào, nên tập nút không đổi. ■

PYTHON

```python
def dao(L):
    truoc, p = None, L.dau
    while p is not None:
        sau = p.ke          # giữ lại phần đuôi trước khi cắt
        p.ke = truoc        # đảo chiều liên kết
        truoc, p = p, sau
    L.dau = truoc
```

C++

```cpp
void dao(DSLK &L) {
    Nut *truoc = nullptr, *p = L.dau;
    while (p != nullptr) {
        Nut *sau = p->ke;   // giữ lại phần đuôi trước khi cắt
        p->ke = truoc;      // đảo chiều liên kết
        truoc = p;
        p = sau;
    }
    L.dau = truoc;
}
```

JULIA

```julia
function dao!(L::DSLK)
    truoc = nothing
    p = L.dau
    while p !== nothing
        sau = p.ke
        p.ke = truoc
        truoc = p
        p = sau
    end
    L.dau = truoc
    return L
end
```

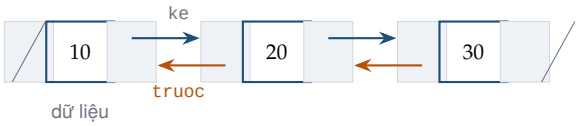
Biến sau ở dòng 3 là bắt buộc: dòng 4 ghi đè `p.ke`, nên nếu không lưu lại trước thì không còn cách nào đi tiếp.

**Ví dụ 5.1.** Danh sách $10 \to 20 \to 30$ sau khi đảo trở thành $30 \to 20 \to 10$. Chương trình kiểm thử chạy 3000 kịch bản ngẫu nhiên gồm các thao tác thêm đầu, thêm cuối, xoá, tìm và đảo, đối chiếu từng bước với danh sách chuẩn của ngôn ngữ; kết quả trùng khớp hoàn toàn ở cả ba bản cài đặt.

### 5.3.6 Danh sách liên kết kép

Danh sách liên kết kép cho mỗi nút hai liên kết: một trỏ tới nút sau, một trỏ tới nút trước. Nhờ đó có thể duyệt ngược, và có thể xoá một nút mà không cần biết nút đứng trước nó.

PYTHON




**Hình 5.4.** Danh sách liên kết kép: mỗi nút có hai liên kết.

```python
class NutKep:
    __slots__ = ("gt", "truoc", "ke")

    def __init__(self, gt):
        self.gt = gt
        self.truoc = None
        self.ke = None

class DSLKKep:
    def __init__(self):
        self.dau = None
        self.cuoi = None        # danh sách kép giữ thêm con trỏ nút cuối
        self.n = 0

def xoa_nut_kep(L, p):
    """Xoá nút p mà không cần duyệt tìm nút đứng trước."""
    if p.truoc is not None:
        p.truoc.ke = p.ke
    else:
        L.dau = p.ke                # p là nút đầu
    if p.ke is not None:
        p.ke.truoc = p.truoc
    else:
        L.cuoi = p.truoc            # p là nút cuối
    L.n -= 1
```

C++

```cpp
struct NutKep {
    int gt;
    NutKep *truoc = nullptr, *ke = nullptr;
    NutKep(int g) : gt(g) {}
};

struct DSLKKep {
    NutKep *dau = nullptr, *cuoi = nullptr;
    int n = 0;
};

void xoa_nut_kep(DSLKKep &L, NutKep *p) {
    if (p->truoc) p->truoc->ke = p->ke;
    else          L.dau = p->ke;        // p là nút đầu
    if (p->ke)    p->ke->truoc = p->truoc;
    else          L.cuoi = p->truoc;    // p là nút cuối
    delete p;
    L.n--;
}
```

JULIA

```julia
mutable struct NutKep
    gt::Int
    truoc::Union{NutKep, Nothing}
    ke::Union{NutKep, Nothing}
end
NutKep(gt) = NutKep(gt, nothing, nothing)

mutable struct DSLKKep
    dau::Union{NutKep, Nothing}
    cuoi::Union{NutKep, Nothing}
    n::Int
end
DSLKKep() = DSLKKep(nothing, nothing, 0)

function xoa_nut_kep!(L::DSLKKep, p::NutKep)
    if p.truoc !== nothing
        p.truoc.ke = p.ke
    else
        L.dau = p.ke
    end
    if p.ke !== nothing
        p.ke.truoc = p.truoc
    else
        L.cuoi = p.truoc
    end
    L.n -= 1
end
```

**Bảng 5.4.** So sánh danh sách liên kết đơn và kép.

| | Đơn | Kép |
|---|---|---|
| Bộ nhớ cho liên kết mỗi nút | 1 con trỏ | 2 con trỏ |
| Duyệt xuôi | $O(n)$ | $O(n)$ |
| Duyệt ngược | không được | $O(n)$ |
| Xoá nút khi đã biết địa chỉ | $O(n)$ | $O(1)$ |
| Số phép gán khi chèn | 2 | 4 |

**Bài 5.1.** Cài đặt lớp danh sách liên kết đơn đầy đủ bằng ba ngôn ngữ, gồm các thao tác thêm đầu, thêm cuối, chèn sau, xoá theo giá trị, tìm kiếm, đảo ngược và in. Viết chương trình kiểm thử tự động: sinh ngẫu nhiên một dãy thao tác, thực hiện song song trên danh sách liên kết của mình và trên danh sách chuẩn của ngôn ngữ, so sánh trạng thái sau mỗi thao tác. Chạy 3000 kịch bản. Riêng bản C++, kiểm tra rò bộ nhớ bằng `valgrind` hoặc bằng cách đếm số lần `new` và `delete`.

## 5.4 Ngăn xếp

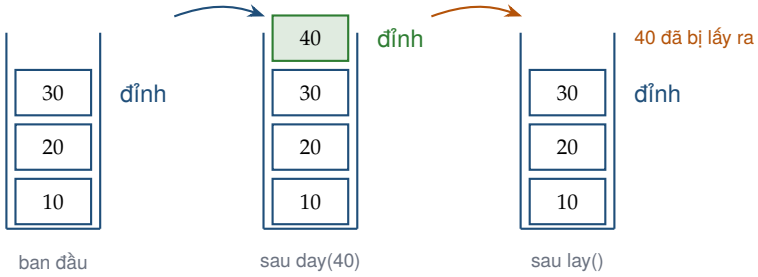
### 5.4.1 Định nghĩa

**Định nghĩa 5.4.** (*Ngăn xếp*) **Ngăn xếp** (*stack*) là danh sách trong đó việc thêm và lấy phần tử chỉ được thực hiện ở *một đầu*, gọi là đỉnh. Phần tử vào sau cùng là phần tử ra trước tiên, nên ngăn

xếp còn gọi là cấu trúc LIFO (*Last In First Out*).

Bốn thao tác cơ bản:

- `day(x)` — đưa $x$ vào đỉnh;
- `lay()` — lấy và trả về phần tử ở đỉnh;
- `dinh()` — xem phần tử ở đỉnh mà không lấy ra;
- `rong()` — kiểm tra ngăn xếp có rỗng không.




**Hình 5.5.** Ba trạng thái của ngăn xếp. Mọi thao tác chỉ xảy ra ở đỉnh.

### 5.4.2 Biểu diễn bằng mảng

Cách đơn giản và nhanh nhất: dùng một mảng và một biến `dinh` ghi chỉ số phần tử trên đỉnh.

JULIA

```julia
mutable struct NganXep
    a::Vector{Int}
end
NganXep() = NganXep(Int[])

day!(s::NganXep, x)  = (push!(s.a, x); s)
lay!(s::NganXep)     = isempty(s.a) ? nothing : pop!(s.a)
dinh(s::NganXep)     = isempty(s.a) ? nothing : s.a[end]
rong(s::NganXep)     = isempty(s.a)
```

PYTHON

```python
class NganXep:
    def __init__(self):
        self.a = []

    def day(self, x):
        self.a.append(x)

    def lay(self):
        return self.a.pop() if self.a else None

    def dinh(self):
        return self.a[-1] if self.a else None

    def rong(self):
        return len(self.a) == 0
```

C++

```cpp
struct NganXep {
    vector<int> a;
    void day(int x)  { a.push_back(x); }
    bool rong()      { return a.empty(); }
    int  dinh()      { return a.back(); }        // gọi khi biết chắc không rỗng
    int  lay()       { int x = a.back(); a.pop_back(); return x; }
};

// Thư viện chuẩn đã có sẵn: stack<int> s;  s.push(x);  s.top();  s.pop();
```

> **Chú ý.** Trong C++, `std::stack::pop()` *không* trả về giá trị; nó chỉ xoá phần tử trên đỉnh. Muốn lấy giá trị phải gọi `top()` trước rồi mới `pop()`. Ngoài ra gọi `top()` trên ngăn xếp rỗng là hành vi không xác định, không phải lỗi được báo. Luôn kiểm tra `empty()` trước.

Cả bốn thao tác đều là $O(1)$.

### 5.4.3 Ứng dụng 1: kiểm tra dấu ngoặc

> **Bài toán.** Cho một xâu ký tự chứa các loại ngoặc `()`, `[]`, `{}` xen lẫn các ký tự khác. Kiểm tra các ngoặc có khớp nhau không, tức mỗi ngoặc mở có đúng một ngoặc đóng cùng loại, và các cặp ngoặc lồng nhau chứ không cắt nhau.

**Ý tưởng.** Quan sát then chốt là: khi gặp một ngoặc đóng, nó bắt buộc phải khớp với ngoặc mở *gần nhất chưa được đóng*. Chẳng hạn trong `([{`, ngoặc đóng đầu tiên phải là `}`. Vậy các ngoặc mở cần được lưu lại theo đúng luật vào sau ra trước, và đó chính là đặc tả của ngăn xếp ở mục 5.1.

**Thuật toán 5.4.** *Kiểm tra dấu ngoặc*

1. Khởi tạo một ngăn xếp rỗng.
2. Duyệt xâu từ trái sang phải, với mỗi ký tự $c$:
   (a) Nếu $c$ là ngoặc mở thì đưa $c$ vào ngăn xếp.
   (b) Nếu $c$ là ngoặc đóng: nếu ngăn xếp rỗng thì trả về *sai*; ngược lại lấy phần tử ở đỉnh ra và nếu nó không cùng loại với $c$ thì trả về *sai*.
   (c) Nếu $c$ không phải ngoặc thì bỏ qua.
3. Hết xâu, trả về *đúng* khi và chỉ khi ngăn xếp rỗng.

GIẢ MÃ

```
CAN-NGOAC(s)
1  st ← ngăn xếp rỗng
2  for mỗi ký tự c trong s
3      if c là ngoặc mở
4          đẩy c vào st
5      else if c là ngoặc đóng
6          if st rỗng
7              return SAI                  // đóng mà chưa từng mở
8          t ← lấy đỉnh st
9          if t không cùng loại với c
```

```
10            return SAI                  // đóng nhầm loại, ngoặc cắt nhau
11 return (st rỗng)                       // còn ngoặc mở chưa đóng thì SAI
```

**Giải thích từng bước.**

| Dòng | Ý nghĩa và lý do có dòng này |
|---|---|
| 6–7 | Bắt trường hợp ngoặc đóng xuất hiện khi chưa có ngoặc mở nào, ví dụ xâu `)(`. Thiếu kiểm tra này, lệnh lấy đỉnh ở dòng 8 chạy trên ngăn xếp rỗng và gây lỗi. |
| 8–10 | Bắt trường hợp ngoặc lồng nhau chéo, ví dụ `(a+[b)*c]`. Lấy đỉnh ra rồi so loại là đủ, không cần duyệt lại toàn bộ ngăn xếp. |
| 11 | Bắt trường hợp còn ngoặc mở chưa đóng, ví dụ `((`. Nếu bỏ dòng này mà trả về *đúng* luôn thì xâu `(()` bị nhận nhầm là hợp lệ. Ba dòng 7, 10, 11 tương ứng với ba kiểu sai khác nhau, và thiếu dòng nào thì chương trình bỏ sót đúng kiểu sai đó. |

**Định lý 5.3.** (*Tính đúng đắn*) *Thuật toán trả về* đúng *khi và chỉ khi xâu vào có các ngoặc khớp nhau.*

*Chứng minh.* **Bất biến.** Sau khi xử lý $i$ ký tự đầu, nội dung ngăn xếp từ đáy lên đỉnh đúng bằng dãy các ngoặc mở chưa được đóng trong $i$ ký tự đó, theo đúng thứ tự xuất hiện.

*Khởi tạo.* Với $i = 0$ chưa có ngoặc nào, và ngăn xếp rỗng.

*Duy trì.* Nếu ký tự thứ $i + 1$ là ngoặc mở, nó trở thành ngoặc mở chưa đóng mới nhất và thuật toán đưa nó lên đỉnh. Nếu là ngoặc đóng hợp lệ, nó đóng đúng ngoặc mở mới nhất, tức phần tử ở đỉnh, và thuật toán lấy phần tử đó ra. Nếu không phải ngoặc thì cả hai vế đều không đổi.

*Kết thúc.* Hết xâu, xâu hợp lệ khi và chỉ khi không còn ngoặc mở nào chưa đóng, tức khi và chỉ khi ngăn xếp rỗng — đúng điều kiện ở dòng 11. Còn trong trường hợp thuật toán thoát sớm ở dòng 7 hoặc 10, ta đã chỉ ra một ngoặc đóng không khớp, nên xâu chắc chắn không hợp lệ. ■

**PYTHON**

```python
def can_ngoac(s):
    cap = {")": "(", "]": "[", "}": "{"}
    st = []
    for c in s:
        if c in "([{":
            st.append(c)
        elif c in ")]}":
            if not st or st[-1] != cap[c]:
                return False
            st.pop()
    return len(st) == 0          # còn ngoặc mở chưa đóng thì sai
```

**JULIA**

```julia
function can_ngoac(s::AbstractString)
    cap = Dict(')' => '(', ']' => '[', '}' => '{')
    st = Char[]
    for c in s
        if c in "([{"
            push!(st, c)
```

```
        elseif c in ")]}"
            (isempty(st) || st[end] != cap[c]) && return false
            pop!(st)
        end
    end
    return isempty(st)
end
```

C++

```
bool can_ngoac(const string &s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')' && t != '(') || (c == ']' && t != '[') ||
                (c == '}' && t != '{')) return false;
        }
    }
    return st.empty();
}
```

**Ví dụ 5.2.** *Kết quả kiểm thử.*

| Biểu thức | Kết quả |
|---|---|
| `(a+[b*c])-{d/e}` | đúng |
| `(a+[b)*c]` | sai (ngoặc lồng nhau chéo) |
| `(((())))` | đúng |
| `(()` | sai (còn một ngoặc mở) |
| xâu rỗng | đúng |
| `a+b` | đúng |

### 5.4.4 Ứng dụng 2: biểu thức hậu tố

Biểu thức viết theo cách quen thuộc gọi là **trung tố**, chẳng hạn $3 + 4 \times 2$. Cách viết này cần quy tắc ưu tiên phép toán và cần dấu ngoặc. Cách viết **hậu tố** đặt phép toán sau hai toán hạng: $3\ 4\ 2\ \times\ +$. Nó không cần dấu ngoặc, không cần quy tắc ưu tiên, và tính được bằng một ngăn xếp duy nhất. Vì vậy nhiều máy ảo và máy tính bỏ túi dùng dạng này.

#### 5.4.4.1 Chuyển trung tố sang hậu tố

Thuật toán mang tên Dijkstra, thường gọi là thuật toán sân ga.

GIẢ MÃ

```
TRUNG-TO-SANG-HAU-TO(bt)
1  ra ← rỗng;  st ← ngăn xếp rỗng
2  for mỗi ký hiệu c trong bt
3      if c là toán hạng:      thêm c vào ra
4      else if c = "(":        đẩy c vào st
```

```
5      else if c = ")":
6          while đỉnh st ≠ "(":  lấy đỉnh st, thêm vào ra
7          bỏ dấu "(" khỏi st
8      else                                    // c là phép toán
9          while st khác rỗng và đỉnh ≠ "(" và
10            (ưu_tiên(đỉnh) > ưu_tiên(c) hoặc
11             (ưu_tiên(đỉnh) = ưu_tiên(c) và c kết hợp từ trái sang phải))
12            lấy đỉnh st, thêm vào ra
13         đẩy c vào st
14 lấy hết st, thêm vào ra
15 return ra
```

PYTHON

```python
UU_TIEN = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}
PHAI_SANG_TRAI = {"^"}              # luỹ thừa kết hợp từ phải sang trái

def trung_to_sang_hau_to(bt):
    ra, st = [], []
    for c in bt.split():
        if c.isalnum():
            ra.append(c)
        elif c == "(":
            st.append(c)
        elif c == ")":
            while st and st[-1] != "(":
                ra.append(st.pop())
            st.pop()                        # bỏ dấu "("
        else:
            while st and st[-1] != "(" and (
                    UU_TIEN[st[-1]] > UU_TIEN[c] or
                    (UU_TIEN[st[-1]] == UU_TIEN[c]
                     and c not in PHAI_SANG_TRAI)):
                ra.append(st.pop())
            st.append(c)
    while st:
        ra.append(st.pop())
    return " ".join(ra)
```

### 5.4.4.2 Tính giá trị biểu thức hậu tố

GIẢ MÃ

```
TINH-HAU-TO(bt)
1  st ← ngăn xếp rỗng
2  for mỗi ký hiệu c trong bt
3      if c là số:  đẩy c vào st
4      else
5          b ← lấy đỉnh st        // toán hạng PHẢI ra trước
6          a ← lấy đỉnh st        // toán hạng TRÁI ra sau
7          đẩy (a c b) vào st
8  return đỉnh st
```

PYTHON

```python
def tinh_hau_to(bt):
    st = []
    for c in bt.split():
        if c.isdigit():
            st.append(int(c))
        else:
            b = st.pop()                    # toán hạng PHẢI ra trước
            a = st.pop()                    # toán hạng TRÁI ra sau
            if   c == "+": st.append(a + b)
            elif c == "-": st.append(a - b)
            elif c == "*": st.append(a * b)
            elif c == "^": st.append(a ** b)
            else:          st.append(int(a / b))   # cắt về 0, giống C++
    return st[-1]
```

C++

```cpp
long long luy_thua_nguyen(long long a, long long n) {
    long long kq = 1;
    while (n-- > 0) kq *= a;
    return kq;
}

long long tinh_hau_to(const string &bt) {
    stringstream ss(bt);
    string tok;
    stack<long long> st;
    while (ss >> tok) {
        if (isdigit(tok[0])) st.push(stoll(tok));
        else {
            long long b = st.top(); st.pop();    // toán hạng PHẢI ra trước
            long long a = st.top(); st.pop();
            st.push(tok == "+" ? a + b : tok == "-" ? a - b
                  : tok == "*" ? a * b
                  : tok == "^" ? luy_thua_nguyen(a, b)
                  : a / b);                      // a / b cắt về 0
        }
    }
    return st.top();
}
```

JULIA

```julia
function tinh_hau_to(bt::AbstractString)
    st = Int[]
    for c in split(bt)
        if all(isdigit, c)
            push!(st, parse(Int, c))
        else
            b = pop!(st)                    # toán hạng PHẢI ra trước
            a = pop!(st)
            push!(st, c == "+" ? a + b : c == "-" ? a - b :
                      c == "*" ? a * b : c == "^" ? a^b :
                      Int(trunc(a / b)))           # cắt về 0, giống C++
        end
    end
    return st[end]
end
```

**Phép chia nguyên trong ba ngôn ngữ.** Toán tử `//` của Python làm tròn *xuống*, còn `/` của C++ và `÷` của Julia cắt phần thập phân về phía 0. Hai cách chỉ khác nhau khi số bị chia âm: $-7//2 = -4$ trong Python nhưng $-7/2 = -3$ trong C++. Vì vậy bản Python ở trên dùng `int(a / b)` để ba bản cài đặt cho cùng kết quả trên mọi dữ liệu vào.

**Thứ tự lấy hai toán hạng.** Toán hạng lấy ra *trước* là toán hạng *bên phải*, vì nó vào ngăn xếp sau. Viết ngược lại thì phép cộng và phép nhân vẫn đúng do có tính giao hoán, nhưng phép trừ và phép chia sai. Đây là lỗi rất khó phát hiện nếu chỉ kiểm thử bằng biểu thức chỉ có cộng và nhân.

**Ví dụ 5.3.**

| Trung tố | Hậu tố | Giá trị |
|---|---|---|
| `3 + 4 * 2 - ( 1 + 1 )` | `3 4 2 * + 1 1 + -` | 9 |
| `( 2 + 3 ) * ( 4 - 1 )` | `2 3 + 4 1 - *` | 15 |
| `5 * ( 6 + 2 ) - 12 / 4` | `5 6 2 + * 12 4 / -` | 37 |
| `2 ^ 3 ^ 2` | `2 3 2 ^ ^` | 512 |

Cả bốn kết quả trùng với giá trị tính trực tiếp bằng biểu thức của ngôn ngữ. Dòng cuối cho thấy phép luỹ thừa được xử lý đúng theo luật kết hợp từ phải sang trái: $2^{(3^2)} = 2^9 = 512$ chứ không phải $(2^3)^2 = 64$.

### 5.4.5 Ứng dụng 3: khử đệ quy

Mọi thuật toán đệ quy đều viết lại được bằng vòng lặp cộng một ngăn xếp, vì bản thân cơ chế đệ quy của ngôn ngữ cũng dùng ngăn xếp lời gọi. Kỹ thuật này cần đến khi độ sâu đệ quy vượt giới hạn của ngôn ngữ, như đã nêu ở mục 3.4.

PYTHON

```python
def duyet_sau_de_quy(cay, goc):
    if goc is not None:
        print(goc, end=" ")
        for con in cay[goc]:
            duyet_sau_de_quy(cay, con)

def duyet_khu_de_quy(cay, goc):
    st = [goc]
    while st:
        u = st.pop()
        print(u, end=" ")
        for con in reversed(cay[u]):    # đảo để thứ tự duyệt không đổi
            st.append(con)
```

Chú thích ở dòng cuối là chi tiết dễ bỏ sót: ngăn xếp lấy ra theo thứ tự ngược với thứ tự đưa vào, nên muốn giữ nguyên thứ tự duyệt của bản đệ quy, phải đẩy các con vào theo thứ tự ngược lại.

## 5.5 Hàng đợi

### 5.5.1 Định nghĩa

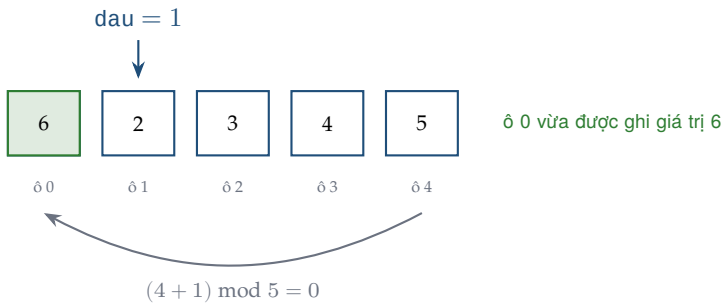
**Định nghĩa 5.5.** (*Hàng đợi*) **Hàng đợi** (*queue*) là danh sách trong đó phần tử được thêm ở một đầu và lấy ra ở đầu kia. Phần tử vào trước là phần tử ra trước, nên hàng đợi còn gọi là cấu trúc FIFO (*First In First Out*).

Hàng đợi mô hình hoá đúng hình ảnh xếp hàng: người đến trước được phục vụ trước. Nó xuất hiện ở mọi nơi cần xử lý theo thứ tự đến: hàng đợi in ấn, hàng đợi yêu cầu tới máy chủ, và quan trọng nhất với học phần này là thuật toán tìm kiếm theo chiều rộng ở chương 6.

### 5.5.2 Cách cài đặt sai và cách cài đặt đúng

Cách cài đặt trực tiếp bằng mảng có một vấn đề. Nếu lấy phần tử ra bằng cách xoá phần tử đầu mảng thì mọi phần tử còn lại phải dịch lên một ô, tốn $O(n)$. Nếu chỉ tăng chỉ số dau lên mà không dịch, thì phần đầu mảng bị bỏ phí và mảng sẽ đầy dù thực tế còn rất ít phần tử.

Cách khắc phục là **hàng đợi vòng**: coi mảng như một vòng tròn, ô cuối nối liền ô đầu. Phép toán lấy phần dư đảm nhiệm việc "quay vòng".




**Hình 5.6.** Hàng đợi vòng sức chứa 5 sau khi thêm $1..5$, lấy ra một phần tử, rồi thêm giá trị 6. Giá trị mới quay về ô 0.

**Bài toán.** Cài đặt hàng đợi sức chứa tối đa $n$ phần tử trên một mảng cố định, sao cho cả hai thao tác thêm vào cuối và lấy ở đầu đều tốn $O(1)$, và mảng không bị bỏ phí sau nhiều lần thêm rồi lấy.

**Thuật toán 5.5.** *Hàng đợi vòng*

Dùng ba đại lượng: mảng $a$ gồm $n$ ô, chỉ số dau của phần tử đầu hàng, và số phần tử hiện có so.

1. **Thêm $x$.** Nếu so bằng $n$ thì hàng đầy, từ chối. Ngược lại ghi $x$ vào ô có chỉ số $(\text{dau} + \text{so}) \bmod n$, rồi tăng so lên 1.
2. **Lấy.** Nếu so bằng 0 thì hàng rỗng, trả về giá trị báo không có. Ngược lại đọc giá trị ở ô dau, đặt $\text{dau} \leftarrow (\text{dau} + 1) \bmod n$, giảm so đi 1 rồi trả về giá trị đã đọc.

**Vì sao ba đại lượng này là đủ.** Ô chứa phần tử thứ $k$ tính từ đầu hàng luôn là ô $(\text{dau}+k) \bmod n$. Phép lấy phần dư làm chỉ số tự quay về 0 khi vượt quá ô cuối, nên mảng được dùng như một vòng tròn và không có ô nào bị bỏ phí.
Phần tử cuối hàng nằm ở ô $(\text{dau} + \text{so} - 1) \bmod n$, nên ô trống tiếp theo là $(\text{dau} + \text{so}) \bmod n$

— đúng biểu thức dùng ở bước thêm. Hàng đầy khi so $= n$ và rỗng khi so $= 0$, hai điều kiện phân biệt rõ ràng.

**PYTHON**

```python
class HangDoiVong:
    def __init__(self, n):
        self.a = [None] * n
        self.n = n
        self.dau = 0                # vị trí phần tử đầu hàng
        self.so = 0                 # số phần tử hiện có

    def them(self, x):
        if self.so == self.n:
            return False            # hàng đầy
        self.a[(self.dau + self.so) % self.n] = x
        self.so += 1
        return True

    def lay(self):
        if self.so == 0:
            return None             # hàng rỗng
        x = self.a[self.dau]
        self.dau = (self.dau + 1) % self.n
        self.so -= 1
        return x
```

**JULIA**

```julia
mutable struct HangDoiVong
    a::Vector{Union{Int, Nothing}}
    n::Int
    dau::Int        # đánh chỉ số từ 0 cho tiện phép chia dư
    so::Int
end
HangDoiVong(n) = HangDoiVong(fill(nothing, n), n, 0, 0)

function them!(q::HangDoiVong, x)
    q.so == q.n && return false
    q.a[(q.dau + q.so) % q.n + 1] = x       # +1 vì Julia đánh số từ 1
    q.so += 1
    return true
end

function lay!(q::HangDoiVong)
    q.so == 0 && return nothing
    x = q.a[q.dau + 1]
    q.dau = (q.dau + 1) % q.n
    q.so -= 1
    return x
end
```

**C++**

```cpp
struct HangDoiVong {
    vector<int> a;
    int n, dau = 0, so = 0;
```

```
    HangDoiVong(int n) : a(n), n(n) {}

    bool them(int x) {
        if (so == n) return false;              // hàng đầy
        a[(dau + so) % n] = x;
        so++;
        return true;
    }
    bool lay(int &x) {
        if (so == 0) return false;              // hàng rỗng
        x = a[dau];
        dau = (dau + 1) % n;
        so--;
        return true;
    }
};

// Thư viện chuẩn: queue<int> q;  q.push(x);  q.front();  q.pop();
```

**Vì sao lưu `so` chứ không lưu `cuoi`.** Nhiều tài liệu cài hàng đợi vòng bằng hai chỉ số `dau` và `cuoi`. Cách đó gặp một vấn đề: khi hàng rỗng và khi hàng đầy, cả hai đều cho `dau == cuoi`, nên không phân biệt được. Người ta phải hy sinh một ô trống hoặc thêm một biến cờ. Cách lưu số phần tử hiện có tránh hẳn vấn đề đó và dễ đọc hơn.

**Ví dụ 5.4.** *Lần vết.* Hàng đợi vòng sức chứa 5:

| Thao tác | Nội dung mảng | Trạng thái |
|---|---|---|
| thêm 1..5 | `[1, 2, 3, 4, 5]` | đầy, `dau` $= 0$ |
| thêm 6 | `[1, 2, 3, 4, 5]` | bị từ chối, hàng đầy |
| lấy | `[1, 2, 3, 4, 5]` | trả về 1, `dau` $= 1$ |
| thêm 6 | `[6, 2, 3, 4, 5]` | ghi vào ô 0, quay vòng |

Sau bước cuối, hàng đợi chứa các giá trị $2, 3, 4, 5, 6$ theo đúng thứ tự vào. Giá trị 1 vẫn còn trong ô 0 cho tới khi bị ghi đè, nhưng không còn thuộc hàng đợi.

Cả hai thao tác đều là $O(1)$, và bộ nhớ cố định $O(n)$ không phụ thuộc số lần thêm.

**Bài 5.2.** Cài đặt ngăn xếp và hàng đợi vòng bằng ba ngôn ngữ, mỗi cấu trúc theo hai cách: bằng mảng và bằng danh sách liên kết. Viết chương trình kiểm thử so sánh với `stack` và queue của thư viện chuẩn trên 3000 dãy thao tác ngẫu nhiên. Sau đó cài chương trình chuyển trung tố sang hậu tố và tính giá trị, kiểm tra với hai biểu thức trong ví dụ và với mười biểu thức tự đặt có chứa phép trừ và phép chia.

## 5.6 Ứng dụng: bộ nhớ phát lại trong học tăng cường

### 5.6.1 Bài toán

Trong học tăng cường, một tác tử tương tác liên tục với môi trường. Mỗi bước sinh ra một **kinh nghiệm**: trạng thái hiện tại, hành động đã chọn, phần thưởng nhận được, trạng thái mới. Mạng nơ-ron của tác tử được huấn luyện từ các kinh nghiệm đó.

Nếu huấn luyện ngay trên kinh nghiệm vừa sinh ra, việc học gặp hai vấn đề.

1. **Các mẫu liên tiếp tương quan mạnh với nhau.** Trạng thái ở bước $t$ và bước $t+1$ gần như giống hệt nhau, nên gradient tính từ chúng chỉ đi theo một hướng và việc học rất kém ổn định.
2. **Mỗi kinh nghiệm chỉ được dùng một lần.** Trong khi việc thu thập kinh nghiệm là phần tốn kém nhất của quá trình huấn luyện.

Giải pháp là **bộ nhớ phát lại** (*experience replay buffer*): lưu các kinh nghiệm gần đây vào một bộ đệm, và mỗi bước huấn luyện thì lấy ngẫu nhiên một lô mẫu từ bộ đệm đó. Việc lấy ngẫu nhiên phá vỡ tương quan; việc lưu lại cho phép mỗi kinh nghiệm được dùng nhiều lần.

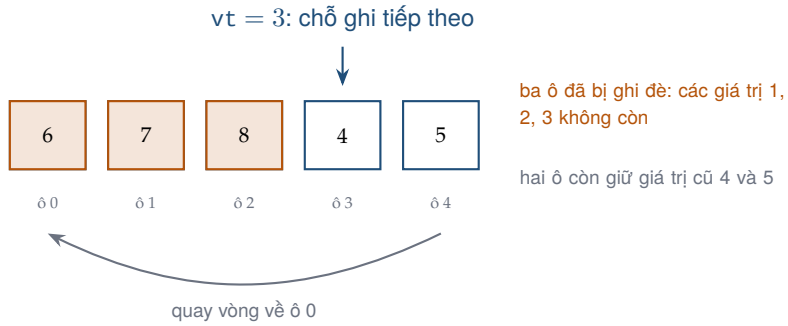
### 5.6.2 Yêu cầu đối với cấu trúc dữ liệu

Bộ đệm phải đáp ứng ba yêu cầu.

1. Thêm một kinh nghiệm mới trong $O(1)$, vì thao tác này chạy ở mỗi bước của môi trường.
2. Khi đầy, tự động loại kinh nghiệm cũ nhất, cũng trong $O(1)$. Đây đúng là hành vi FIFO của hàng đợi.
3. Lấy ngẫu nhiên phần tử thứ $i$ trong $O(1)$, để lấy một lô mẫu nhanh.

Yêu cầu thứ ba loại bỏ danh sách liên kết, vì truy cập theo chỉ số trên danh sách liên kết tốn $O(n)$; bảng số liệu ở mục 5.2 cho thấy khác biệt lên tới ba mươi nghìn lần. Yêu cầu thứ hai loại bỏ mảng thường, vì xoá phần tử đầu mảng tốn $O(n)$.

Cấu trúc thoả cả ba yêu cầu là *mảng vòng*, tức chính hàng đợi vòng ở mục 5.5, với một thay đổi nhỏ: khi đầy thì ghi đè lên phần tử cũ nhất thay vì từ chối.




**Hình 5.7.** Bộ nhớ phát lại sức chứa 5 sau khi thêm lần lượt các giá trị $1..8$. Ba giá trị cũ nhất đã bị ghi đè.

### 5.6.3 Cài đặt

**Thuật toán 5.6.** *Bộ nhớ phát lại trên mảng vòng*

Dùng mảng $a$ gồm $n$ ô, chỉ số `vt` của ô sẽ ghi tiếp theo, và số phần tử thực có `so`.

1. **Thêm một kinh nghiệm $x$.** Ghi $x$ vào ô `vt`, bất kể ô đó đang trống hay đang chứa dữ liệu cũ. Đặt $\texttt{vt} \leftarrow (\texttt{vt}+1) \bmod n$ và $\texttt{so} \leftarrow \min(\texttt{so}+1,\ n)$.
2. **Lấy một lô $k$ mẫu.** Chọn ngẫu nhiên $k$ chỉ số phân biệt trong khoảng $0$ tới $\texttt{so}-1$ rồi trả về các phần tử tương ứng.

Khác biệt duy nhất so với hàng đợi vòng ở mục 5.5 nằm ở bước thêm: hàng đợi *từ chối* khi

đầy, còn bộ nhớ phát lại *ghi đè* lên phần tử cũ nhất.

**GIẢ MÃ**

```
THEM(x)
1  a[vt] ← x
2  vt ← (vt + 1) mod n              // quay vòng
3  so ← min(so + 1, n)              // số phần tử thực có

LAY-MAU(k)
1  chọn ngẫu nhiên k chỉ số phân biệt trong 0..so-1
2  return các phần tử tương ứng
```

### Giải thích từng bước.

| Dòng | Ý nghĩa và lý do có dòng này |
|---|---|
| 1 | Ghi đè không điều kiện. Ô `vt` luôn là ô chứa kinh nghiệm cũ nhất khi bộ nhớ đã đầy, vì con trỏ ghi chạy vòng đều đặn; nên việc ghi đè đúng bằng việc loại bỏ phần tử cũ nhất, và tốn $O(1)$. |
| 2 | Phép lấy phần dư đưa con trỏ ghi quay về ô 0 sau khi qua ô cuối. |
| 3 | Số phần tử thực có tăng dần cho tới khi bằng sức chứa rồi dừng ở đó. Biến này cần thiết vì trong giai đoạn đầu, phần đuôi mảng còn chưa có dữ liệu; lấy mẫu trên toàn mảng khi đó sẽ trả về các ô rỗng. |

**PYTHON**

```python
import random

class BoNhoPhatLai:
    def __init__(self, suc_chua):
        self.a = [None] * suc_chua
        self.n = suc_chua
        self.vt = 0                 # vị trí sẽ ghi tiếp theo
        self.so = 0                 # số phần tử thực có

    def them(self, kinh_nghiem):
        self.a[self.vt] = kinh_nghiem
        self.vt = (self.vt + 1) % self.n        # quay vòng, ghi đè cái cũ nhất
        self.so = min(self.so + 1, self.n)

    def lay_mau(self, k):
        return random.sample(self.a[:self.so], k)

    def du_de_hoc(self, k):
        return self.so >= k
```

**JULIA**

```julia
mutable struct BoNhoPhatLai
    a::Vector{Any}
    n::Int
    vt::Int         # đánh chỉ số từ 0 cho tiện phép chia dư
    so::Int
```

```
end
BoNhoPhatLai(suc_chua) = BoNhoPhatLai(Vector{Any}(undef, suc_chua),
                                      suc_chua, 0, 0)

using Random            # randperm nằm trong thư viện chuẩn Random

function them!(b::BoNhoPhatLai, kinh_nghiem)
    b.a[b.vt + 1] = kinh_nghiem
    b.vt = (b.vt + 1) % b.n
    b.so = min(b.so + 1, b.n)
    return b
end

lay_mau(b::BoNhoPhatLai, k) = b.a[randperm(b.so)[1:k]]
du_de_hoc(b::BoNhoPhatLai, k) = b.so >= k
```

C++

```
struct KinhNghiem {
    vector<double> trang_thai, trang_thai_moi;
    int hanh_dong;
    double phan_thuong;
    bool ket_thuc;
};

struct BoNhoPhatLai {
    vector<KinhNghiem> a;
    int n, vt = 0, so = 0;
    mt19937 rng{12345};

    BoNhoPhatLai(int suc_chua) : a(suc_chua), n(suc_chua) {}

    void them(const KinhNghiem &kn) {
        a[vt] = kn;
        vt = (vt + 1) % n;              // quay vòng, ghi đè cái cũ nhất
        so = min(so + 1, n);
    }

    vector<KinhNghiem> lay_mau(int k) {
        vector<int> chi_so(so);
        iota(chi_so.begin(), chi_so.end(), 0);
        shuffle(chi_so.begin(), chi_so.end(), rng);
        vector<KinhNghiem> lo;
        for (int i = 0; i < k; i++) lo.push_back(a[chi_so[i]]);
        return lo;
    }

    bool du_de_hoc(int k) const { return so >= k; }
};
```

**Ví dụ 5.5.** *Kết quả đo.* Bộ nhớ sức chứa 5, thêm lần lượt các giá trị $1, 2, \ldots, 8$. Trạng thái cuối cùng của mảng là `[6, 7, 8, 4, 5]`, với `vt` $= 3$ và `so` $= 5$. Ba ô đầu đã bị ghi đè bởi các giá trị 6, 7, 8; hai ô cuối vẫn giữ 4 và 5. Nội dung logic của bộ nhớ là năm kinh nghiệm gần nhất $\{4, 5, 6, 7, 8\}$, đúng như yêu cầu.

**Thứ tự trong mảng không phải thứ tự thời gian.** Trong ví dụ trên, mảng vật lý là `[6, 7, 8, 4, 5]` nhưng thứ tự thời gian là $4, 5, 6, 7, 8$. Với bộ nhớ phát lại, điều này không quan trọng, vì mẫu được lấy *ngẫu nhiên* nên thứ tự vô nghĩa. Nhưng nếu dùng cùng cấu trúc để lấy "$k$ kinh nghiệm gần nhất theo thứ tự", phải khôi phục thứ tự bằng phép chia dư chứ không được đọc thẳng mảng.

**Bảng 5.5.** Vì sao mảng vòng là lựa chọn duy nhất thoả cả ba yêu cầu.

| Cấu trúc | Thêm mới | Loại cái cũ nhất | Truy cập ngẫu nhiên |
|---|---|---|---|
| Mảng động, xoá đầu | $O(1)$ | $O(n)$ | $O(1)$ |
| Danh sách liên kết đơn | $O(1)$ | $O(1)$ | $O(n)$ |
| Danh sách liên kết kép | $O(1)$ | $O(1)$ | $O(n)$ |
| **Mảng vòng** | $O(1)$ | $O(1)$ | $O(1)$ |

**Nhận xét 5.4.** Kích thước bộ nhớ phát lại là một tham số quan trọng của thuật toán học. Bộ nhớ quá nhỏ thì các mẫu vẫn còn tương quan; bộ nhớ quá lớn thì tác tử học từ những kinh nghiệm đã lỗi thời, sinh ra trong giai đoạn nó còn hành động rất kém. Các bản cài đặt phổ biến dùng sức chứa từ $10^5$ đến $10^6$ kinh nghiệm. Với $10^6$ kinh nghiệm, việc chọn cấu trúc dữ liệu không còn là chi tiết kỹ thuật: một thao tác $O(n)$ chạy ở mỗi bước sẽ khiến toàn bộ quá trình huấn luyện không thể hoàn thành.

**Bài 5.3.** Cài đặt bộ nhớ phát lại theo các bước sau.

1. Định nghĩa kiểu dữ liệu kinh nghiệm gồm bốn trường: trạng thái, hành động, phần thưởng, trạng thái mới.
2. Cài lớp `BoNhoPhatLai` với ba phương thức `them`, `lay_mau`, `du_de_hoc`.
3. Kiểm thử: tạo bộ nhớ sức chứa 5, thêm các số $1..8$, in nội dung mảng. Kết quả phải là `[6, 7, 8, 4, 5]`.
4. Viết một môi trường giả lập đơn giản, chẳng hạn tác tử đi trên đoạn thẳng từ 0 đến 10 và nhận phần thưởng khi tới đích. Chạy 10 000 bước, mỗi bước thêm một kinh nghiệm và lấy một lô 32 mẫu.
5. Đo thời gian chạy khi bộ nhớ được cài bằng mảng vòng và khi được cài bằng danh sách Python thường có `pop(0)`. Với sức chứa $10^5$, chênh lệch phải rất rõ.

**Tóm tắt.** Mảng và danh sách liên kết là hai cách cài đặt danh sách với ưu nhược điểm trái ngược nhau: mảng truy cập theo chỉ số trong $O(1)$ nhưng chèn đầu tốn $O(n)$; danh sách liên kết thì ngược lại. Số đo thực tế cho chênh lệch tới 255 lần theo chiều này và 30 000 lần theo chiều kia. Ngăn xếp là danh sách chỉ thao tác ở một đầu, phù hợp với mọi bài toán có cấu trúc lồng nhau như kiểm tra ngoặc, tính biểu thức hậu tố và khử đệ quy. Hàng đợi thao tác ở hai đầu và phải cài bằng mảng vòng để đạt $O(1)$. Bộ nhớ phát lại trong học tăng cường là một hàng đợi vòng có ghi đè, và ba yêu cầu của nó chỉ có mảng vòng đáp ứng được cả ba.

## 5.7 Bài tập chương 5

### Câu hỏi ôn tập

1. So sánh mảng và danh sách liên kết trên bốn thao tác: truy cập theo chỉ số, tìm theo giá trị, chèn đầu, xoá nút đã biết địa chỉ.
2. Vì sao chênh lệch thời gian truy cập ngẫu nhiên giữa mảng và danh sách liên kết lớn hơn nhiều so với dự đoán từ ký hiệu $O$?
3. Trong phép chèn sau một nút, vì sao thứ tự hai phép gán là bắt buộc? Điều gì xảy ra nếu đảo thứ tự?
4. Vì sao xoá một nút khỏi danh sách liên kết đơn cần biết nút đứng trước? Danh sách liên kết kép khắc phục điều này thế nào?
5. Trong thuật toán đảo danh sách, vai trò của biến `sau` là gì?
6. Trong C++, hai lỗi nào liên quan tới `delete` khi xoá nút?
7. Phát biểu định nghĩa ngăn xếp và giải thích tên gọi LIFO.
8. Vì sao bài toán kiểm tra dấu ngoặc phù hợp với ngăn xếp?
9. Trong thuật toán tính biểu thức hậu tố, vì sao toán hạng lấy ra trước lại là toán hạng bên phải? Với phép toán nào thì viết sai vẫn cho kết quả đúng?
10. Vì sao hàng đợi cài bằng mảng thường lại kém hiệu quả? Hàng đợi vòng khắc phục thế nào?
11. Vì sao nên lưu số phần tử hiện có thay vì lưu chỉ số cuối trong hàng đợi vòng?
12. Nêu ba yêu cầu của bộ nhớ phát lại và giải thích vì sao chỉ mảng vòng thoả cả ba.

### Bài tập tính toán

13. Vẽ sơ đồ danh sách liên kết đơn sau dãy thao tác: thêm cuối 5, thêm cuối 9, thêm đầu 2, xoá 9, thêm cuối 7, đảo.
14. Cho danh sách $1 \to 2 \to 3 \to 4 \to 5$. Vẽ trạng thái sau mỗi vòng lặp của thuật toán đảo.
15. Lần vết ngăn xếp khi kiểm tra biểu thức `{[(a+b)*c]-[d/(e-f)]}`.
16. Chuyển các biểu thức sau sang hậu tố và tính giá trị: `5 * ( 6 + 2 ) - 12 / 4` và `2 ^ 3 ^ 2` (phép luỹ thừa kết hợp từ phải sang trái).
17. Lần vết hàng đợi vòng sức chứa 4 với dãy thao tác: thêm 1, thêm 2, lấy, thêm 3, thêm 4, thêm 5, lấy, thêm 6. Ghi nội dung mảng, `dau` và `so` sau mỗi bước.
18. Bộ nhớ phát lại sức chứa 4, thêm các giá trị $1..11$. Xác định nội dung mảng, giá trị `vt` và tập kinh nghiệm còn giữ.

### Bài tập lập trình mức cơ bản

19. Cài đặt danh sách liên kết đơn đầy đủ bằng ba ngôn ngữ theo bài thực hành 5.1.
20. Viết hàm đếm số nút của danh sách liên kết bằng hai cách: dùng biến đếm cập nhật khi thêm hoặc xoá, và duyệt lại từ đầu. So sánh độ phức tạp.

21. Viết hàm tìm nút ở giữa danh sách liên kết chỉ bằng một lượt duyệt, dùng kỹ thuật hai con trỏ chạy với tốc độ khác nhau.
22. Viết hàm kiểm tra một danh sách liên kết có chứa chu trình không, dùng thuật toán hai con trỏ nhanh chậm của Floyd.
23. Viết hàm trộn hai danh sách liên kết đã sắp xếp thành một danh sách đã sắp xếp, không cấp phát nút mới.
24. Cài đặt ngăn xếp bằng danh sách liên kết và bằng mảng, so sánh thời gian với 10 triệu thao tác.
25. Cài đặt hàng đợi bằng hai ngăn xếp, mỗi thao tác đạt $O(1)$ khi tính trung bình.
26. Viết chương trình kiểm tra một xâu có phải xâu đối xứng không, dùng ngăn xếp.

### Bài tập lập trình mức vận dụng

27. Cài đặt danh sách liên kết vòng và viết chương trình giải bài toán Josephus: $n$ người đứng thành vòng tròn, đếm đến người thứ $k$ thì loại, tiếp tục cho tới khi còn một người.
28. Cài đặt một trình soạn thảo văn bản đơn giản với chức năng hoàn tác và làm lại, dùng hai ngăn xếp.
29. Cài đặt máy tính bỏ túi đọc biểu thức trung tố có ngoặc và bốn phép toán, chuyển sang hậu tố rồi tính. Xử lý cả trường hợp biểu thức sai cú pháp.
30. Cài đặt cấu trúc `MinStack`: ngăn xếp có thêm thao tác lấy giá trị nhỏ nhất trong $O(1)$. Gợi ý: dùng thêm một ngăn xếp phụ.
31. Cài đặt hàng đợi hai đầu (*deque*) bằng mảng vòng, cho phép thêm và lấy ở cả hai đầu trong $O(1)$.
32. Cho một dãy số và cửa sổ trượt độ dài $k$. In giá trị lớn nhất trong mỗi cửa sổ, tổng thời gian $O(n)$, dùng hàng đợi hai đầu.
33. Cài đặt bài toán tìm phần tử lớn hơn kế tiếp: với mỗi phần tử của mảng, tìm phần tử đầu tiên bên phải lớn hơn nó, tổng thời gian $O(n)$ bằng ngăn xếp.

### Bài tập phần trí tuệ nhân tạo

34. Cài đặt bộ nhớ phát lại theo bài thực hành 5.3 và xác nhận kết quả `[6, 7, 8, 4, 5]`.
35. Đo thời gian thêm $10^6$ kinh nghiệm vào bộ nhớ sức chứa $10^5$ với ba cách cài đặt: mảng vòng, danh sách Python với `pop(0)`, và `collections.deque` có tham số `maxlen`. Lập bảng kết quả.
36. Cài đặt *phát lại có ưu tiên*: mỗi kinh nghiệm có một trọng số, và xác suất được chọn tỉ lệ với trọng số đó. Cài bản đơn giản $O(n)$ mỗi lần lấy mẫu, rồi giải thích vì sao bản thực dụng cần cấu trúc cây tổng, một biến thể của đống ở chương 7.
37. Cài đặt hàng đợi dùng cho thuật toán tìm kiếm theo chiều rộng: đọc một lưới có tường, tìm đường ngắn nhất từ ô xuất phát tới ô đích. So sánh kết quả với chương 6 sau khi học xong chương đó.
38. Viết chương trình mô phỏng một hệ thống hàng đợi yêu cầu: các yêu cầu đến theo thời gian ngẫu nhiên, mỗi yêu cầu cần một thời gian xử lý ngẫu nhiên. Đo thời gian chờ trung bình khi số máy chủ thay đổi từ 1 đến 5.